# UNICESUMAR

**Aula 007 - Expressões, Atribuição e Estruturas de Controle**

**Aluno:** Matheus Costa e Silva — **RA:** 24000729-2

**Maringá**

**02/10/2026**

## Página 4 — Quiz de Abertura: Análise do Console

*Respostas esperadas conforme execução do código nas linguagens (Python, Java, JavaScript, C):*

1. **Python**: -4 512
2. **Java**: 1245
3. **JavaScript**: "33" / true false true
4. **C**: Exibe valores com base na formatação e atribuições especificadas.
5. **Python**: padrão True
6. **JavaScript**: true false true

## Página 5 — Expressões Aritméticas: Aspectos de Projeto

A avaliação de expressões aritméticas depende das regras de precedência, associatividade e da ordem de avaliação dos operandos. Compreender esses fatores é crucial para evitar ambiguidades no código.

- **Perguntas centrais de projeto:**
  - Quais são as hierarquias de precedência?
  - Qual a orientação da associatividade?
  - Como é determinada a ordem de avaliação dos operandos?
  - Existem limites para efeitos colaterais?
  - É permitido sobrecarregar operadores?
  - Quais combinações de tipos são suportadas?
- **Lisp e Notação Prefixa:** Lisp elimina prioridade de operadores utilizando notação prefixa explícita entre parênteses:
  (+ a (* b c))
- **Avaliação de** $a + b * c$ **(onde** $a = 3, b = 4, c = 5$**):**
  - *Esquerda para a direita:* $(3 + 4) * 5 = 35$
  - *Direita para a esquerda:* $3 + (4 * 5) = 23$
  - *Com precedência padrão ($$ antes de +):** $3 + 4 * 5 = 23$
  - *APL (sempre da direita para a esquerda):* $3 * 4 + 5 = 27$

## Página 6 — Regras de Precedência e Associatividade na Prática

A precedência indica qual operação deve ser resolvida primeiro, enquanto a associatividade resolve empates entre operadores de mesma prioridade.

| Expressão | Python | JavaScript | Java |
|---|---|---|---|
| -2 ** 2 | -4 | Erro de sintaxe | *(sem operador **)* |
| 2 ** 3 ** 2 | 512 | 512 | — |
| 1 + 2 + "3" | TypeError | "33" | "33" |
| -7 / 2 (inteiros) | -4 (com //) | -3.5 | -3 |
| -7 % 2 | 1 | -1 | -1 |

- **Direções de Associatividade:**
    - *À esquerda:* *, /, %, +, -
    - *À direita:* Exponenciação ** (Python, JS, Ruby), incremento/decremento ++, --, e menos unário.
    - *Excel:* --2^2 resulta em 4 (o operador unário possui maior prioridade).

**Boa prática:** Na dúvida, utilize parênteses para deixar a intenção do cálculo clara para o compilador e para quem for ler o código.

## Página 7 — Efeitos Colaterais e Ordem dos Operandos

Quando funções modificam variáveis externas durante o cálculo de uma expressão, a ordem em que os operandos são lidos pode alterar completamente o resultado.

- **Exemplo de Efeito Colateral:**
- C

```
int a = 5;
int fun1(void) {
   a = 17; // Altera o valor global de 'a'
   return 3;
}
int main(void) {
   a = a + fun1(); // O resultado depende de 'a' ser lido antes ou depois da chamada
}
```

```[cite: 4]

- 
- 
- **Comportamento por Linguagem:**
    - **Java:** Garante avaliação estrita da esquerda para a direita (resultado: 8).
    - **C (gcc):** Não define ordem fixa pelo padrão (pode resultar em 20).
    - **Go:** A especificação não garante a ordem de leitura de a em relação à execução da função (resultado: 20).
- **Transparência Referencial:** Um programa é referencialmente transparente quando a substituição de uma expressão pelo seu valor retornado não altera o comportamento. Sem efeitos colaterais, a ordem de avaliação torna-se irrelevante.

## Página 8 — Sobrecarga de Operadores

Permite que o mesmo símbolo desempenhe ações distintas dependendo dos tipos dos operandos envolvidos (ex: + somando números ou concatenando textos).

- **Pontos de Atenção:**
    - Pode introduzir ambiguidades no código (ex: em C++, & representa o operando *E bit a bit* e também a obtenção de endereço).
    - **Suportado pelo usuário:** C++, C#, Python, Rust e Kotlin.
    - **Não suportado pelo usuário:** Java.
- **Exemplo em Python (Métodos Mágicos):**
- Python

```
class Vetor:
    def __init__(self, x, y):
        self.x, self.y = x, y
    def __add__(self, outro):
        return Vetor(self.x + outro.x, self.y + outro.y)
    def __mul__(self, k):
        return Vetor(self.x * k, self.y * k)

v = Vetor(1, 2) + Vetor(3, 4) # Vetor(4, 6)
print(v * 3)                  # Vetor(12, 18)
```[cite: 5]

- 
- 

## Página 9 — Conversões de Tipos: Alargamento, Estreitamento e Coerção

Converte valores entre tipos incompatíveis, podendo ocorrer de forma explícita (cast) ou implícita (coerção).

- **Alargamento vs. Estreitamento:**
```

    - *Alargamento:* Converte para um tipo com maior capacidade (geralmente seguro, embora possa haver perda marginal de precisão).
    - *Estreitamento:* Converte para um tipo de menor capacidade, podendo truncar dados.
- **Regras por Linguagem:**
    - **C / Java:** Permitem coerções automáticas em certas situações.
    - **Go / Rust / ML:** Exigem conversões estritamente explícitas (geram erro de compilação caso haja mistura).

**Aviso:** A coerção implícita pode mascarar erros de lógica durante a compilação.

## Página 10 — Coerções Implícitas em JavaScript e PHP

Linguagens fracamente tipadas realizam coerções automáticas agressivas que podem resultar em comportamentos não intuitivos.

- **Exemplos em JavaScript:**
    - "5" - 2 → 3 (converte para número)
    - "5" + 2 → "52" (concatena como string)
    - true + 1 → 2
    - 0 == "0" → true *(mas 0 === "0" → false)*
- **PHP ("Type Juggling"):**
    - Comparação com == converte strings numéricas automaticamente, gerando inconsistências em comparações de hashes.

**Recomendação:** Em JS e PHP, priorize o uso dos operadores estritos === e !== para evitar coerções indesejadas.

## Página 11 — Expressões Relacionais e Booleanas

Servem para comparar valores e estruturar condições de tomada de decisão.

- **Interpretação Booleana:**
    - Em **C**, não havia tipo booleano nativo original; qualquer valor diferente de zero é tratado como verdadeiro (if (-1) executa).
    - Em **Java, C#, Go e Rust**, o bloco if exige estritamente uma expressão que retorne um booleano.
- **Comparações Encadeadas em Python:**
    - Expressões como a > b > c são válidas e avaliadas como (a > b) and (b > c).

## Página 12 — Avaliação em Curto-Circuito

A avaliação é interrompida assim que o resultado final da expressão lógica for determinado.

- **Aplicações Práticas:**
    - Evita exceções de ponteiro nulo: while (i < lista.length && lista[i] != chave).
    - Em Java, && realiza curto-circuito, enquanto & avalia ambos os lados obrigatoriamente.

    - Em JavaScript, os operadores de encadeamento opcional ?. e de coalescência nula ?? facilitam a navegação e definição de valores padrão sem gerar erros de runtime.

## Página 13 — Comandos de Atribuição

Associa o resultado de uma expressão a um identificador de memória.

- **Atribuição como Expressão:**
    - Em **C, C++ e JS**, a atribuição retorna um valor, permitindo atribuições encadeadas (a = b = 0) ou dentro de estruturas condicionais (if (x = y)).
    - Em **Python, Go e Rust**, a atribuição é uma instrução (statement), prevenindo erros acidentais de atribuição dentro de um if.
- **Recursos Modernos:**
    - Python oferece atribuição múltipla (a, b = b, a) e o operador morsa := para atribuições em expressões.
    - Linguagens funcionais usam vinculações de imutabilidade (val ou let).

## Página 14 — Atribuição de Modo Misto

Ocorre quando os tipos da variável de destino e do valor atribuído diferem.

- **Atenção:** O cálculo do lado direito é resolvido *antes* de ocorrer a conversão para o tipo da variável de destino.
- **Armadilha Frequente:**
- Java

double taxa = acertos / total * 100;

-
- Se acertos e total forem inteiros, a divisão será inteira (descartando as casas decimais para 0) antes da conversão para double.

## Página 15 — Estruturas de Controle de Fluxo

Propostas pelo **Teorema de Böhm-Jacopini (1966)**, apenas três estruturas fundamentais são necessárias para construir qualquer algoritmo:

1. **Sequência:** Execução linear de instruções.
2. **Seleção:** Tomada de decisão baseada em condição (if / else).
3. **Iteração:** Repetição baseada em condição (while).

Estruturas adicionais (for, switch, do-while, foreach) foram criadas para melhorar a expressividade e legibilidade do código.

## Página 16 — Seleção de Dois Caminhos: O Problema do "Else Pendente"

Em estruturas condicionais aninhadas sem delimitação clara por chaves, surge a dúvida sobre a qual if um bloco else pertence.

- **Regra Geral (C, C++, Java, C#):** O else é associado ao if mais interno e próximo que ainda não possua um else correspondente, independentemente da indentação.
- **Solução:** Linguagens modernas como Rust, Go e Perl tornam o uso de chaves {} obrigatório, eliminando o problema.

## Página 17 — Seleção Múltipla: Do Switch ao Match Exaustivo

Permite ramificar o fluxo entre múltiplas opções.

- **Switch Tradicional (C):** Sem a instrução break, a execução "cai" para os casos subsequentes (*fallthrough*).
- **Switch Moderno (Java 14+ / Go):** Não exige break explícito e pode funcionar como uma expressão que retorna valor.
- **Pattern Matching (Rust):** O comando match exige a verificação exaustiva de todas as possibilidades em tempo de compilação.

## Página 18 — Laços de Repetição Controlados por Contador

Estruturas de iteração com limite definido.

- **Comportamento Diferenciado:**
  - **C:** O for é flexível; alterar a variável de controle dentro do bloco modifica o fluxo da contagem.
  - **Python:** O for percorre um objeto iterável (range); reatribuir a variável do laço no corpo não altera a sequência de iteração original.

## Página 19 — Laços Lógicos e Controle de Interrupção

- **Pré-teste (while):** Valida a condição antes de executar o bloco.
- **Pós-teste (do-while):** Garante ao menos uma execução do bloco antes de testar a condição.
- **Interrupções:** O uso de break (incluindo versões com rótulo em Java) permite sair antecipadamente de laços aninhados.
- **Cuidado:** Falhas no incremento ou atualização da condição causam laços infinitos (ex: caso histórico do *Zune 2008*).

## Página 20 — Desvio Incondicional (goto)

Permite saltar a execução para qualquer ponto rotulado do código.

- **Crítica de Dijkstra (1968):** A publicação *"Go To Statement Considered Harmful"* argumentou que o uso descontrolado do goto prejudica a rastreabilidade e legibilidade do software.
- **Uso Atual:** Banido em Java, mas presente sob restrições em C, C# e Go. Estruturas como break, continue e return cumprem papéis de desvio de forma estruturada.

## Página 21 — Comandos Protegidos de Dijkstra

Conceito em que a execução é associada a expressões guardadas por condições.

- Quando múltiplas guardas avaliam como verdadeiras, a escolha de qual ramo executar ocorre de maneira não-determinística.

- Este princípio inspirou o modelo de concorrência CSP de Hoare e a instrução select na linguagem Go.

## Páginas 22 a 24 — Resolução dos Exercícios de Identificação de Bugs

1. **JavaScript (for...in em Arrays):**
   - *Problema:* O laço for...in itera pelas chaves (índices/strings), resultando em concatenação textual ("0012").
   - *Correção:* Substituir por for...of.
2. **Python (Parâmetro Opcional com Valor Zero):**
   - *Problema:* A expressão d = d or 10 interpreta 0 como valor falso, aplicando indevidamente o desconto de 10.
   - *Correção:* Checar explicitamente if d is None:.
3. **Java (switch sem break):**
   - *Problema:* A ausência de break faz a execução passar por todos os casos subsequentes até o default, atribuindo "inválido".
   - *Correção:* Adicionar a instrução break ao fim de cada caso ou utilizar a nova sintaxe case ->.
4. **C (Ponto e Vírgula Indevido no if):**
   - *Problema:* O ; logo após o if (saque > saldo); encerra a instrução condicional, fazendo o printf subsequente executar sempre.
   - *Correção:* Remover o ponto e vírgula após a condição do if.
5. **Go (Sombreamento de Variável / *Shadowing*):**
   - *Problema:* O operador := dentro do bloco declara uma nova variável local x, mantendo o x externo inalterado.
   - *Correção:* Utilizar atribuição simples = no escopo interno.
6. **Java (Divisão de Inteiros):**
   - *Problema:* acertos / total realiza uma divisão inteira (7 / 10 = 0), resultando em 0.0.
   - *Correção:* Realizar cast para double: (double) acertos / total * 100.

## Página 25 — Resumo: Armadilhas de Sintaxe e Mecanismos de Proteção

| Tema | Armadilha Comum | Mecanismo de Prevenção |
|---|---|---|
| **Precedência** | Ambiguidade em expressões complexas | Uso explícito de parênteses |
| **Operandos** | Efeitos colaterais em chamadas de função | Funções puras e ordem fixa de avaliação |

| | | |
|---|---|---|
| **Conversões** | Coerção implícita de tipos | Tipagem forte e conversões explícitas |
| **Booleanos** | Valoração de tipos não-booleanos em if | Restrição de estruturas condicionais a tipos booleanos |
| **Atribuição** | Confusão entre = e == em condições | Atribuição tratada como instrução e não expressão |
| **Seleção** | Queda acidental de casos em switch / Else pendente | Uso de match exaustivo, case -> e chaves obrigatórias |
| **Laços** | Uso inadequado de for...in / Laços infinitos | Iteradores dedicados (for...of) e checagem de progresso |

## Página 26 — Atividades Complementares

- **Tópicos de Autopesquisa:**
    1. Expressões e avaliação em Ruby e Lisp.
    2. Implementação interna de seletores múltiplos via tabelas de desvio (*jump tables*).
    3. Substituição de laços por recursão em linguagens funcionais.
    4. Leitura do artigo clássico de Edsger W. Dijkstra: *"Go To Statement Considered Harmful"* (1968).